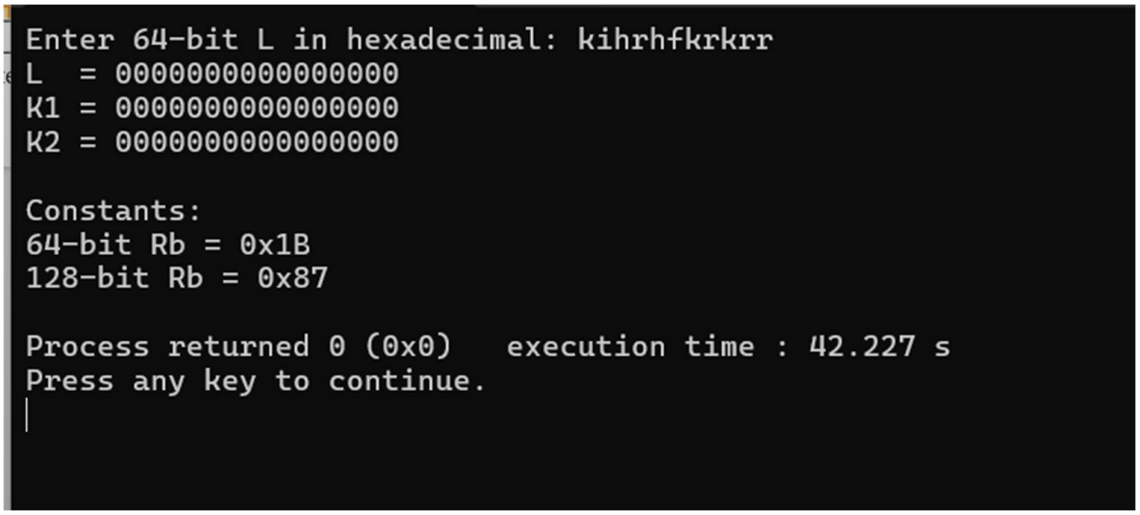
```
31. #include <stdio.h>
#include <stdint.h>

uint64_t leftShift64(uint64_t x)
{
   return x << 1;
}

void generate64(uint64_t L)
{
   uint64_t K1, K2;
   uint64_t Rb = 0x1B;

   if(L & 0x8000000000000000ULL)
      K1 = leftShift64(L) ^ Rb;
   else
      K1 = leftShift64(L);

   if(K1 & 0x8000000000000000ULL)
      K2 = leftShift64(K1) ^ Rb;
   else
      K2 = leftShift64(K1);

   printf("L  = %016llX\n", L);
   printf("K1 = %016llX\n", K1);
   printf("K2 = %016llX\n", K2);
}

int main()
{
   uint64_t L;

   printf("Enter 64-bit L in hexadecimal: ");
   scanf("%llx", &L);

   generate64(L);

   printf("\nConstants:\n");
   printf("64-bit Rb = 0x1B\n");
   printf("128-bit Rb = 0x87\n");

   return 0;
}
```



32. #include <stdio.h>

```
long long modPow(long long a, long long b, long long p)
{
   long long result = 1;

   while(b > 0)
   {
      if(b % 2)
         result = (result * a) % p;

      a = (a * a) % p;
      b /= 2;
   }

   return result;
}

long long modInverse(long long a, long long q)
{
   long long i;

   for(i = 1; i < q; i++)
   {
      if((a * i) % q == 1)
         return i;
   }

   return -1;
}

int main()
```

```
{
    long long p = 23;
    long long q = 11;
    long long g = 4;
    long long x = 3;

    long long y;
    long long h = 7;
    long long k1 = 2;
    long long k2 = 5;

    long long r1, s1;
    long long r2, s2;
    long long kinv;

    y = modPow(g, x, p);

    r1 = modPow(g, k1, p) % q;
    kinv = modInverse(k1, q);

    s1 = (kinv * (h + x * r1)) % q;

    r2 = modPow(g, k2, p) % q;
    kinv = modInverse(k2, q);

    s2 = (kinv * (h + x * r2)) % q;

    printf("Public key y = %lld\n", y);

    printf("\nSignature 1:\n");
    printf("k = %lld\n", k1);
    printf("r = %lld\n", r1);
    printf("s = %lld\n", s1);

    printf("\nSignature 2:\n");
    printf("k = %lld\n", k2);
    printf("r = %lld\n", r2);
    printf("s = %lld\n", s2);

    if(r1 != r2 || s1 != s2)
        printf("\nSame message produced different signatures.\n");

    return 0;
}
```

```
Public key y = 18

Signature 1:
k = 2
r = 5
s = 0

Signature 2:
k = 5
r = 1
s = 2

Same message produced different signatures.

Process returned 0 (0x0)   execution time : 1.023 s
Press any key to continue.
```

33.
```
#include <stdio.h>
#include <stdint.h>

uint32_t feistel(uint32_t r, uint32_t key)
{
   r ^= key;
   r = (r << 1) | (r >> 31);
   r ^= 0xA5A5A5A5;

   return r;
}

uint64_t desEncrypt(uint64_t block, uint64_t key)
{
   uint32_t L = block >> 32;
   uint32_t R = block & 0xFFFFFFFF;
   uint32_t temp;
   int i;

   for(i = 0; i < 16; i++)
   {
      temp = R;
      R = L ^ feistel(R, (uint32_t)(key >> (i % 32)));
      L = temp;
   }

   return ((uint64_t)R << 32) | L;
}
```

```
uint64_t desDecrypt(uint64_t block, uint64_t key)
{
   uint32_t L = block >> 32;
   uint32_t R = block & 0xFFFFFFFF;
   uint32_t temp;
   int i;

   for(i = 15; i >= 0; i--)
   {
      temp = L;
      L = R ^ feistel(L, (uint32_t)(key >> (i % 32)));
      R = temp;
   }

   return ((uint64_t)L << 32) | R;
}

int main()
{
   uint64_t plaintext;
   uint64_t key;
   uint64_t ciphertext;
   uint64_t decrypted;

   printf("Enter 64-bit plaintext in hexadecimal: ");
   scanf("%llx", &plaintext);

   printf("Enter key in hexadecimal: ");
   scanf("%llx", &key);

   ciphertext = desEncrypt(plaintext, key);

   decrypted = desDecrypt(ciphertext, key);

   printf("Plaintext  : %016llX\n", plaintext);
   printf("Ciphertext : %016llX\n", ciphertext);
   printf("Decrypted  : %016llX\n", decrypted);

   return 0;
}
```

```
Enter 64-bit plaintext in hexadecimal: jijfjrjflss
Enter key in hexadecimal: Plaintext  : 0000000100000000
Ciphertext : 1A867EE8B22EF4D9
Decrypted  : 6524B7C4EF151F6D

Process returned 0 (0x0)   execution time : 4.647 s
Press any key to continue.
```

34.
```
#include <stdio.h>
#include <string.h>

void pad(char *input, char *output, int blockSize)
{
   int len = strlen(input);
   int padding = blockSize - (len % blockSize);
   int i;

   if(padding == 0)
      padding = blockSize;

   strcpy(output, input);

   for(i = 0; i < padding - 1; i++)
      output[len + i] = '0';

   output[len + padding - 1] = '1';
   output[len + padding] = '\0';
}

void xorBlock(char *a, char *b, char *result, int n)
{
   int i;

   for(i = 0; i < n; i++)
      result[i] = (a[i] == b[i]) ? '0' : '1';

   result[n] = '\0';
}

int main()
{
   char plaintext[100];
   char padded[100];
   char previous[20];
   char result[20];
```

```
int blockSize;
int i;

printf("Enter binary plaintext: ");
scanf("%s", plaintext);

printf("Enter block size: ");
scanf("%d", &blockSize);

pad(plaintext, padded, blockSize);

printf("\nOriginal plaintext : %s\n", plaintext);
printf("Padded plaintext   : %s\n", padded);

printf("\nECB Mode:\n");

for(i = 0; i < strlen(padded); i += blockSize)
{
   printf("Block: ");

   for(int j = i; j < i + blockSize; j++)
      printf("%c", padded[j]);

   printf("\n");
}

printf("\nCBC Mode:\n");

for(i = 0; i < blockSize; i++)
   previous[i] = '0';

previous[blockSize] = '\0';

for(i = 0; i < strlen(padded); i += blockSize)
{
   xorBlock(&padded[i],
            previous,
            result,
            blockSize);

   printf("XOR Block: %s\n", result);

   strcpy(previous, result);
}
```

```
    printf("\nCFB Mode:\n");

    printf("Plaintext is processed using feedback from the previous ciphertext block.\n");

    printf("\nPadding is added even for complete blocks to make\n");
    printf("the padding pattern unambiguous during decryption.\n");

    return 0;
}
```

```
Enter binary plaintext: ehfjefkedlked
Enter block size: 34

Original plaintext : ehfjefkedlked
Padded plaintext   : ehfjefkedlked00000000000000000001

ECB Mode:
Block: ehfjefkedlked00000000000000000001

CBC Mode:
XOR Block: 00111111111110000000000000000001
```

```
35. #include <stdio.h>
#include <string.h>

void pad(char *input, char *output, int blockSize)
{
    int len = strlen(input);
    int padding = blockSize - (len % blockSize);
    int i;

    if(padding == 0)
        padding = blockSize;

    strcpy(output, input);

    for(i = 0; i < padding - 1; i++)
        output[len + i] = '0';

    output[len + padding - 1] = '1';
    output[len + padding] = '\0';
}

void xorBlock(char *a, char *b, char *result, int n)
{
    int i;
```

```
    for(i = 0; i < n; i++)
        result[i] = (a[i] == b[i]) ? '0' : '1';

    result[n] = '\0';
}

int main()
{
    char plaintext[100];
    char padded[100];
    char previous[20];
    char result[20];

    int blockSize;
    int i;

    printf("Enter binary plaintext: ");
    scanf("%s", plaintext);

    printf("Enter block size: ");
    scanf("%d", &blockSize);

    pad(plaintext, padded, blockSize);

    printf("\nOriginal plaintext : %s\n", plaintext);
    printf("Padded plaintext   : %s\n", padded);

    printf("\nECB Mode:\n");

    for(i = 0; i < strlen(padded); i += blockSize)
    {
        printf("Block: ");

        for(int j = i; j < i + blockSize; j++)
            printf("%c", padded[j]);

        printf("\n");
    }

    printf("\nCBC Mode:\n");

    for(i = 0; i < blockSize; i++)
        previous[i] = '0';

    previous[blockSize] = '\0';
```

```
    for(i = 0; i < strlen(padded); i += blockSize)
    {
        xorBlock(&padded[i],
                 previous,
                 result,
                 blockSize);

        printf("XOR Block: %s\n", result);

        strcpy(previous, result);
    }

    printf("\nCFB Mode:\n");

    printf("Plaintext is processed using feedback from the previous ciphertext block.\n");

    printf("\nPadding is added even for complete blocks to make\n");
    printf("the padding pattern unambiguous during decryption.\n");

    return 0;
}
```